

Avaliação de Desempenho na Transferência de Arquivos:

Cliente-Servidor × P2P

Relatório dos experimentos

1. Objetivo

Comparar o tempo de conclusão da transferência de um arquivo de um nó de origem para vários nós clientes usando quatro arquiteturas: três variações de cliente-servidor (sequencial, uma thread por cliente e pool de threads) e uma arquitetura P2P. Os experimentos variam o tamanho do arquivo (5 MB, 50 MB e 500 MB) e o número de clientes. Para cada experimento são reportados os tempos **mínimo, médio e máximo** de conclusão entre os clientes. Nos clientes o arquivo baixado é descartado (não é gravado em disco).

2. Arquiteturas implementadas

Tudo foi implementado em Python 3 (somente biblioteca padrão), sobre sockets TCP.

- **CS sequencial** (*server.py --mode seq*): o servidor atende um download por vez; os demais clientes aguardam na fila de conexões do TCP.
- **CS multithread** (*--mode thread*): uma thread por conexão; todos os clientes são atendidos simultaneamente e dividem a banda do servidor.
- **CS pool** (*--mode pool*): pool com $N = 5$ threads; no máximo 5 clientes são atendidos ao mesmo tempo e os demais esperam na fila do pool.
- **P2P** (*p2p_peer.py*, implementação própria no estilo BitTorrent): o arquivo é dividido em peças (256 KB para 5 MB; 1 MB para 50 e 500 MB). Há 1 seed (peer com o arquivo completo) e N leechers. Cada leecher consulta periodicamente (a cada 0,2 s) o mapa de peças dos demais peers, escolhe a peça mais rara (*rarest-first*, desempate aleatório), prefere buscá-la de outro leecher para poupar o seed e, assim que recebe uma peça, passa a servi-la aos outros. Cada peer atende no máximo 4 uploads simultâneos e usa 4 threads de download. Peers que terminam continuam semeando até o fim do experimento.

3. Metodologia

- **Ambiente:** máquina virtual Linux (Ubuntu 24) com 1 vCPU e 4 GB de RAM; todos os nós são processos distintos comunicando-se por TCP em *localhost*.
- **Emulação de rede:** como o loopback não possui gargalo, a banda de **upload de cada nó** (servidor ou peer) foi limitada a 50 MB/s (400 Mbit/s) por um limitador compartilhado entre todas as conexões do nó, equivalente a uma única placa de rede. O download não é limitado.
- **Sincronização:** todos os clientes aguardam um instante de início comum; o tempo de cada cliente é medido da sua abertura de conexão (ou início do swarm) até receber o último byte. No servidor com fila (sequencial e pool) o tempo de espera na fila faz parte do tempo medido.
- **Variáveis:** tamanho do arquivo (5, 50 e 500 MB) e nº de clientes (1, 5, 10 e 20; para 500 MB, 1, 5 e 10). No P2P, "clientes" = nº de leechers, além de 1 seed.
- **Repetições:** 3 execuções (5 MB), 2 (50 MB) e 1 (500 MB). Em cada execução calcula-se o mínimo, a média e o máximo entre os clientes; as tabelas mostram a média desses valores entre as repetições. Os dados brutos (tempo de cada cliente) estão em *resultados/resultados.csv*.

- **Integridade:** cada cliente verifica que recebeu todos os bytes (o experimento é invalidado caso contrário).

Modelo esperado.

Seja S o tamanho do arquivo e $C = 50$ MB/s a banda de upload. Para $S = 500$ MB, $S/C = 10$ s. No servidor sequencial o k -ésimo cliente termina em $k \cdot S/C$: mínimo S/C , médio $(N+1)/2 \cdot S/C$ e máximo $N \cdot S/C$. No multithread todos terminam juntos em $N \cdot S/C$. No pool de 5 threads os clientes terminam em lotes de 5: com $N = 10$ em $5 \cdot S/C$ e $10 \cdot S/C$ (mínimo 50 s, médio 75 s, máximo 100 s). No P2P o seed precisa enviar apenas uma cópia do arquivo (S/C) e os peers se ajudam, de modo que o tempo cresce muito mais devagar com N .

4. Resultados

Tempos de conclusão da transferência por cliente, em segundos (mínimo, médio e máximo entre os clientes).

4.1 Arquivo de 5 MB (3 repetição(ões) por configuração)

Clientes	Arquitetura	Mínimo (s)	Médio (s)	Máximo (s)
1	CS sequencial (1 por vez)	0.11	0.11	0.11
	CS multithread (todos)	0.11	0.11	0.11
	CS pool (max 5)	0.11	0.11	0.11
	P2P (swarm)	0.12	0.12	0.12
5	CS sequencial (1 por vez)	0.11	0.33	0.55
	CS multithread (todos)	0.48	0.49	0.50
	CS pool (max 5)	0.45	0.48	0.50
	P2P (swarm)	0.28	0.32	0.34
10	CS sequencial (1 por vez)	0.11	0.59	1.07
	CS multithread (todos)	0.93	0.98	1.01
	CS pool (max 5)	0.45	0.73	0.99
	P2P (swarm)	0.36	0.46	0.50
20	CS sequencial (1 por vez)	0.12	1.13	2.12
	CS multithread (todos)	1.75	1.94	2.01
	CS pool (max 5)	0.41	1.22	1.98
	P2P (swarm)	0.53	0.72	0.80

4.2 Arquivo de 50 MB (2 repetição(ões) por configuração)

Clientes	Arquitetura	Mínimo (s)	Médio (s)	Máximo (s)
1	CS sequencial (1 por vez)	1.04	1.04	1.04
	CS multithread (todos)	1.05	1.05	1.05
	CS pool (max 5)	1.05	1.05	1.05
	P2P (swarm)	1.02	1.02	1.02
5	CS sequencial (1 por vez)	1.06	3.14	5.24
	CS multithread (todos)	4.98	4.99	5.00
	CS pool (max 5)	4.96	4.99	5.00
	P2P (swarm)	2.03	2.49	2.67
10	CS sequencial (1 por vez)	1.05	5.73	10.42
	CS multithread (todos)	9.93	9.98	10.01
	CS pool (max 5)	4.94	7.48	9.99
	P2P (swarm)	2.18	3.26	3.66
20	CS sequencial (1 por vez)	1.07	10.97	20.89
	CS multithread (todos)	19.74	19.94	20.00
	CS pool (max 5)	4.92	12.48	19.98
	P2P (swarm)	2.38	5.02	5.50

4.3 Arquivo de 500 MB (1 repetição(ões) por configuração)

Clientes	Arquitetura	Mínimo (s)	Médio (s)	Máximo (s)
1	CS sequencial (1 por vez)	10.43	10.43	10.43
	CS multithread (todos)	10.41	10.41	10.41
	CS pool (max 5)	10.41	10.41	10.41
	P2P (swarm)	10.02	10.02	10.02
5	CS sequencial (1 por vez)	10.43	31.27	52.10
	CS multithread (todos)	49.96	49.98	50.00
	CS pool (max 5)	49.96	49.98	50.00
	P2P (swarm)	17.34	19.56	20.54
10	CS sequencial (1 por vez)	10.48	57.45	104.36
	CS multithread (todos)	99.93	99.97	100.00
	CS pool (max 5)	49.98	75.03	100.05
	P2P (swarm)	15.63	26.98	29.00

4.4 Gráficos

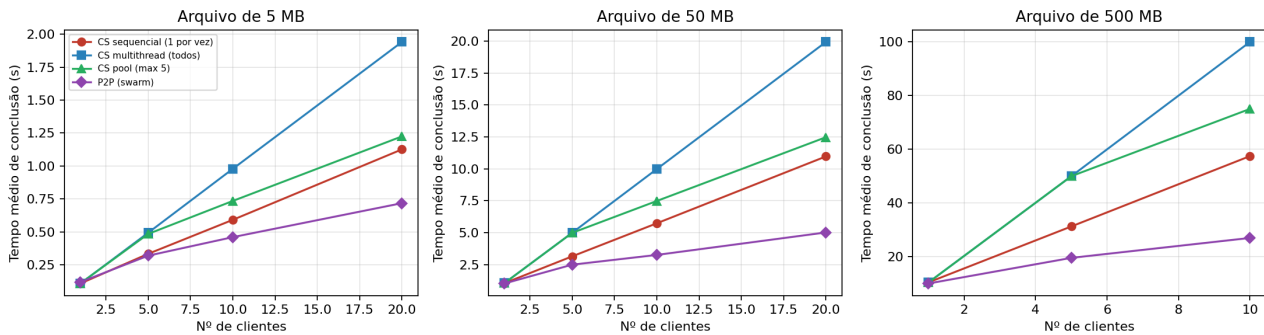


Figura 1 – Tempo médio de conclusão em função do número de clientes.



Figura 2 – Tempos mínimo, médio e máximo para o maior número de clientes de cada tamanho.

5. Análise

- **Um único cliente:** as quatro arquiteturas são equivalentes (500 MB: 10.4 s no CS e 10.0 s no P2P), pois o limite é a banda de upload de um nó.
- **CS sequencial:** melhor tempo mínimo (o primeiro cliente recebe a banda inteira: 10.5 s com 10 clientes), mas o máximo cresce linearmente com N (104.4 s). É justo apenas na ordem de chegada; o último cliente espera por todos.
- **CS multithread:** divide a banda igualmente; mínimo, médio e máximo coincidem (≈ 100 s para 10 clientes e 500 MB). É o pior caso para o tempo médio, pois ninguém termina antes de todos; em compensação, a variação entre clientes é mínima.
- **CS pool:** com $N \leq 5$ comporta-se como o multithread; acima disso, atende em lotes e reduz o tempo médio (75 s contra 100 s) sem alterar o tempo máximo. Também limita o consumo de recursos do servidor (threads e conexões ativas).
- **P2P:** escala muito melhor. Para 500 MB e 10 clientes o máximo foi 29.0 s, contra ≈ 100 s do CS (cerca de 3.4x mais rápido); para 50 MB e 20 clientes, 5.5 s contra 20.0 s. Como cada peer contribui com sua banda de upload, a capacidade total do sistema cresce com o número de nós, ao passo que no CS a banda do servidor é dividida entre todos.
- **Custo do P2P:** o tempo mínimo do P2P é maior que o do sequencial (ex.: 15.6 s contra 10.5 s), pois a banda do seed é compartilhada desde o início. Para arquivos pequenos (5 MB) o ganho é menor, pois a coordenação (consulta de mapas de peças, conexões) pesa mais em relação ao tempo total (0.80 s no P2P contra 2.01 s no CS multithread).
- **Tendência com N:** nos CS o tempo máximo cresce proporcionalmente a $N \cdot S/C$ (ex.: 50 MB: 5, 10 e 20 s para 5, 10 e 20 clientes). No P2P o crescimento é sublinear (50 MB: 2,7, 3,7 e 5,5 s), em linha com o que a literatura prevê para swarms (tempo de distribuição $\approx S/C +$ crescimento logarítmico em N).

6. Limitações

- Os experimentos foram executados em uma única máquina com 1 vCPU (todos os nós disputam a mesma CPU), com banda emulada por software. Os valores absolutos não representam uma rede real (latência, perdas, banda de download e caminhos heterogêneos não são modelados), mas as tendências relativas entre as arquiteturas são válidas.
- Para 500 MB foram usados no máximo 10 clientes: no P2P cada peer guarda as peças em disco temporário para poder re-semeiar, e 20 cópias de 500 MB excederiam o espaço disponível. Também só houve 1 repetição nesse tamanho (os tempos do CS são muito estáveis, mas o P2P tem variação aleatória maior).
- O P2P é uma implementação simplificada (sem tracker, sem tit-for-tat, peers fixos e todos conectados entre si), e não um cliente BitTorrent completo.

7. Conclusão

Com poucos clientes ou arquivos pequenos, o modelo cliente-servidor é simples e suficiente. À medida que o número de clientes e o tamanho do arquivo crescem, o servidor torna-se o gargalo e o tempo de conclusão cresce linearmente com N, qualquer que seja a política de atendimento (sequencial, multithread ou pool); a política apenas redistribui o tempo entre os clientes (sequencial e pool favorecem o tempo médio e mínimo, multithread favorece a equidade). O P2P reduz de forma significativa o tempo máximo e o médio ao aproveitar a banda de upload dos próprios clientes, sendo a melhor opção para distribuir arquivos grandes a muitos nós.

8. Como reproduzir

```
cd src && python3 run_experiments.py          # executa os experimentos (gera
resultados/resultados.csv)
python3 make_report.py    # gera gráficos e este PDF
```